📄 Manual de Integración y Actualización Backend - Proyecto "Activa"

Propósito del Documento: Este manual detalla las implementaciones, modificaciones de arquitectura y especificaciones de la API realizadas en el Backend para soportar el módulo de Gestión de Citas (Appointments) y Autenticación del Fisioterapeuta.

📁 1. Archivos Modificados y Creados (Arquitectura Limpia)

Para alinear el Backend con las necesidades reales del Frontend y la Base de Datos, se intervinieron los siguientes archivos bajo el patrón de Clean Architecture:

Capa de Infraestructura (Controladores y Repositorios)

- **`src/infrastructure/http/controllers/appointment.controller.ts`**
  - **Modificación:** Se actualizó el método `create`. Se eliminó la dependencia de recibir el `id_physio` desde el *body* del Frontend por motivos de seguridad.
  - **Nueva Lógica:** El controlador ahora extrae dinámicamente el `id_physio` desde el token JWT de la sesión (`req.user.id`).
- **`src/infrastructure/persistence/repositories/SequelizeAppointmentRepository.ts`**
  - **Modificación:** Se actualizó el método `findByPhysio` (o equivalente de consulta `GET`).
  - **Nueva Lógica:** Se implementó un *Deep Include (Join)* para que al consultar las citas, Sequelize traiga automáticamente los datos de la tabla `Patient` y, de forma anidada, los datos de la tabla `User` (para obtener el email del paciente).

Capa de Aplicación (Casos de Uso / Lógica de Negocio)

- **`src/application/use-cases/CreateAppointment.uc.ts`**
  - **Modificación:** Se reestructuró el DTO de validación de entrada. Se eliminó el campo fantasma `scheduled_at` y se reemplazó por las columnas reales de la BD: `date`, `start_time` y `end_time`.
  - **Nueva Lógica (Anti-Overlap):** Se programó un algoritmo de validación de horarios. El caso de uso ahora consulta las citas existentes del fisioterapeuta en esa fecha y verifica mediante una fórmula matemática que la nueva cita no choque ni se empalme con una existente. Si choca, lanza un error `400 Bad Request`.

🔌 2. Documentación de Endpoints (Consumo de API)

Todos los endpoints detallados a continuación requieren el envío del Token JWT del Fisioterapeuta en los *Headers* de la petición.

Headers Requeridos (Globales):

JSON
```
{
  "Authorization": "Bearer <TOKEN_JWT_DEL_FISIO>",
  "Content-Type": "application/json"
}
```

🗓️ A. Agendar Nueva Cita

- **Ruta:** `POST /api/appointments` (o la ruta que tengan configurada en el router)
- **Descripción:** Crea una nueva cita médica. El backend asigna automáticamente el `id_physio` basado en el token. Valida que no haya choques de horario.

Body (Request):

JSON
```
{
  "id_patient": 2,
  "date": "2026-04-10",
  "start_time": "10:00:00",
  "end_time": "11:00:00",
  "status": "pending",
  "notes": "Servicio: Valoración Inicial\n\nPaciente reporta dolor
lumbar."
}
```

Respuestas:

- `201 Created`: Cita guardada exitosamente.
- `400 Bad Request`: "Horario no disponible. Se cruza con una cita programada..." (Error de la lógica Anti-Overlap).

💼 B. Obtener Historial y Agenda

- **Ruta:** `GET /api/appointments`
- **Descripción:** Devuelve todas las citas correspondientes al fisioterapeuta logueado. Incluye la información cruzada del paciente y su cuenta de usuario.

Response (`200 OK`):

JSON
```
{
  "rows": [
    {
      "id_appointment": 15,
      "id_physio": 1,
      "id_patient": 2,
      "date": "2026-04-10",
      "start_time": "10:00:00",
      "end_time": "11:00:00",
      "status": "pending",
      "notes": "Servicio: Terapia Física",
      "Patient": {
        "id_patient": 2,
        "first_name": "Gabriel",
        "last_name_paternal": "Domínguez",
        "phone": "2381234567",
        "User": {
          "email": "gabriel@email.com"
        }
      }
    }
  ]
}
```

🔄 C. Reprogramar o Actualizar Estado de Cita

- **Ruta:** `PUT` o `PATCH /api/appointments/:id` (Donde `:id` es el ID de la cita).
- **Descripción:** Modifica el estado de una cita (`completed`, `cancelled`) o sus horarios en caso de una reprogramación.

Body (Request para cambiar estado):

JSON
```
{
  "status": "completed"
}
```

Body (Request para reprogramar):

JSON
```
{
  "date": "2026-04-12",
  "start_time": "16:00:00",
  "end_time": "17:00:00"
}
```

□ 3. Notas Técnicas para el Equipo de Backend

1. **Modelo de Datos (`Patient` y `User`):** Para que el endpoint de obtener citas funcione y envíe el correo al frontend, es **estrictamente necesario** que las asociaciones en Sequelize estén bien definidas. `PatientModel` debe tener un `belongsTo` o `hasOne` con `UserModel`.
2. **Manejo de Errores Frontend:** El Frontend de Angular está programado para leer el atributo `message` de los errores del backend. Asegúrense de que cualquier `throw new Error("Mensaje")` en los Casos de Uso se formatee en el controlador devolviendo `res.status(400).json({ message: err.message }).`
3. **Fechas y Zonas Horarias:** El Frontend envía las fechas en formato ISO crudo (`YYYY-MM-DD`) y las horas en formato de 24 horas (`HH:mm:ss`). Se solicita al backend no aplicar conversiones de TimeZone intermedias que puedan desfasar la cita.